Challenge: Enhancing Earth System Modelling with Artificial Intelligence: Emulators ~~vs Hybrid Models~~

Said Ouala, IMT Atlantique

Emulators of numerical models

- Ocean models are great tools but they are slow

Numerical ocean model

$$\partial_t u = \delta u, \quad \delta u = fv - \frac{1}{\cos\phi}\partial_x p_{hyd} - \frac{1}{\cos\phi}\partial_x p_s + \frac{uv}{a}\tan\phi - \boldsymbol{\nabla}\cdot\boldsymbol{u}u + \partial_z A_v \partial_z u$$

$$\partial_t v = \delta v, \quad \delta v = -fu - \partial_y p_{hyd} - \partial_y p_s - \frac{u^2}{a}\tan\phi + \partial_z A_v \partial_z v - \boldsymbol{\nabla}\cdot\boldsymbol{u}v$$

$$0 = \frac{1}{\cos\phi}\left(\partial_x u + \partial_y(v\cos\phi)\right) + \partial_z w$$

$$0 = p_{hyd} - \int_z^0 dz g\rho/\rho_0$$

$$\partial_t S = \delta S, \quad \delta S = -\boldsymbol{\nabla}\cdot\boldsymbol{u}S + D_S, \quad \boldsymbol{u}\cdot S = \frac{1}{\cos\phi}\left(\partial_x(uS) + \partial_y(vS\cos\phi)\right) + \partial_z(wS)$$

$$\partial_t T = \delta T, \quad \delta T = -\boldsymbol{\nabla}\cdot\boldsymbol{u}T + D_T, \quad \boldsymbol{u}\cdot T = \frac{1}{\cos\phi}\left(\partial_x(uT) + \partial_y(vT\cos\phi)\right) + \partial_z(wT)$$

$$0 = -\left(\partial_x \frac{1}{\cos\phi} h\partial_x p_s + \partial_y h\cos\phi\partial_y p_s\right) + \left(\partial_x \int_{-h}^0 dz\delta u + \partial_y \cos\phi \int_{-h}^0 dz\delta v\right)$$

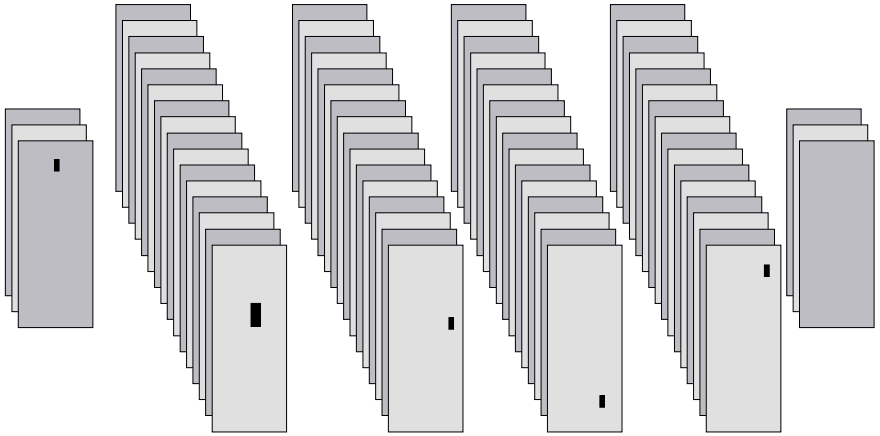
$$\rho = f(p, T, S)$$

- These models are constrained by time stepping
- Climate scale simulations are expensive
- Can not produce large enough ensembles to reliably model the uncertainty due to model imperfections

Emulators of numerical models

- Emulators are not slow, but are they useful ?

AI based emulator



- These models are constrained by time stepping
- Climate scale simulations are expensive
- Can not produce large enough ensembles to reliably model the uncertainty due to model imperfections

Emulators of numerical models

- An emulators in our context is built as follows:

- We start from a differential equation that models the state of the ocean

$$\frac{\partial \mathbf{u}_t}{\partial t} = \mathbf{F}(\mathbf{u}_t) + \mathbf{M}_{\boldsymbol{\theta}}(\cdot)$$

- We use numerical integration to generate a "long enough" time series that we will be using to train the emulator

$$\mathbf{u}_{t+nh} = \Psi^n(\mathbf{u}_t) = \underbrace{\Psi \circ \Psi \circ \cdots \circ \Psi}_{n \text{ times}}(\mathbf{u}_t)$$

- We design an architecture, and the goal is to find:

$$\mathbf{u}_{t+nh} = \Psi^n(\mathbf{u}_t) \approx NN_\theta^n(\mathbf{u}_t)$$

Emulators of numerical models

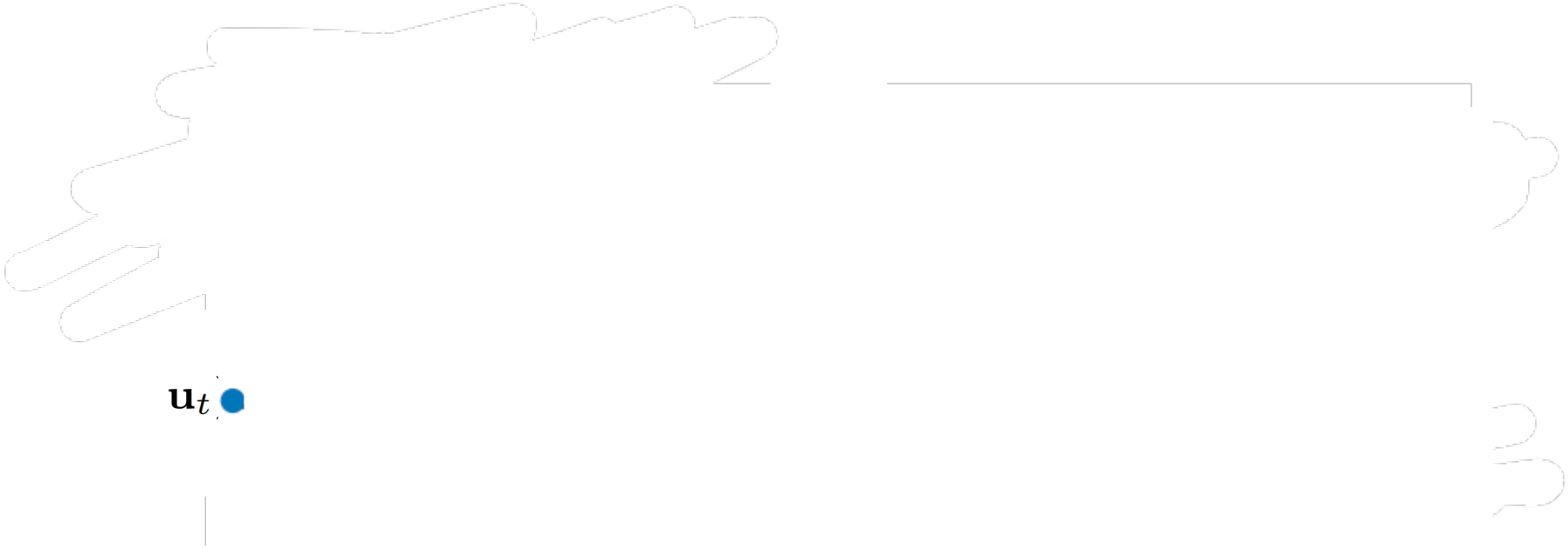
- An emulators in our context is built as follows:

- Optimization objective function:

$$\mathcal{J}(\cdot) = \frac{1}{N} \sum_{k} \frac{1}{n} \sum_{j=1}^{n} \| NN_{\theta}^{j}(\mathbf{u}_{t_k}) - \Psi^{j}(\mathbf{u}_{t_k}) \| + \mathcal{R}(\cdot)$$

Emulators of numerical models

Training visualization



Emulators of numerical models

Training visualization

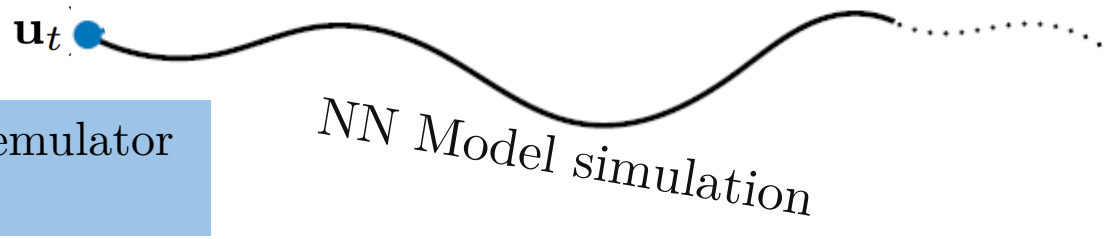
Initial condition : from the trainig data

Emulators of numerical models

Training visualization

Initial condition : from the trainig data

Run the autoregressive emulator forward

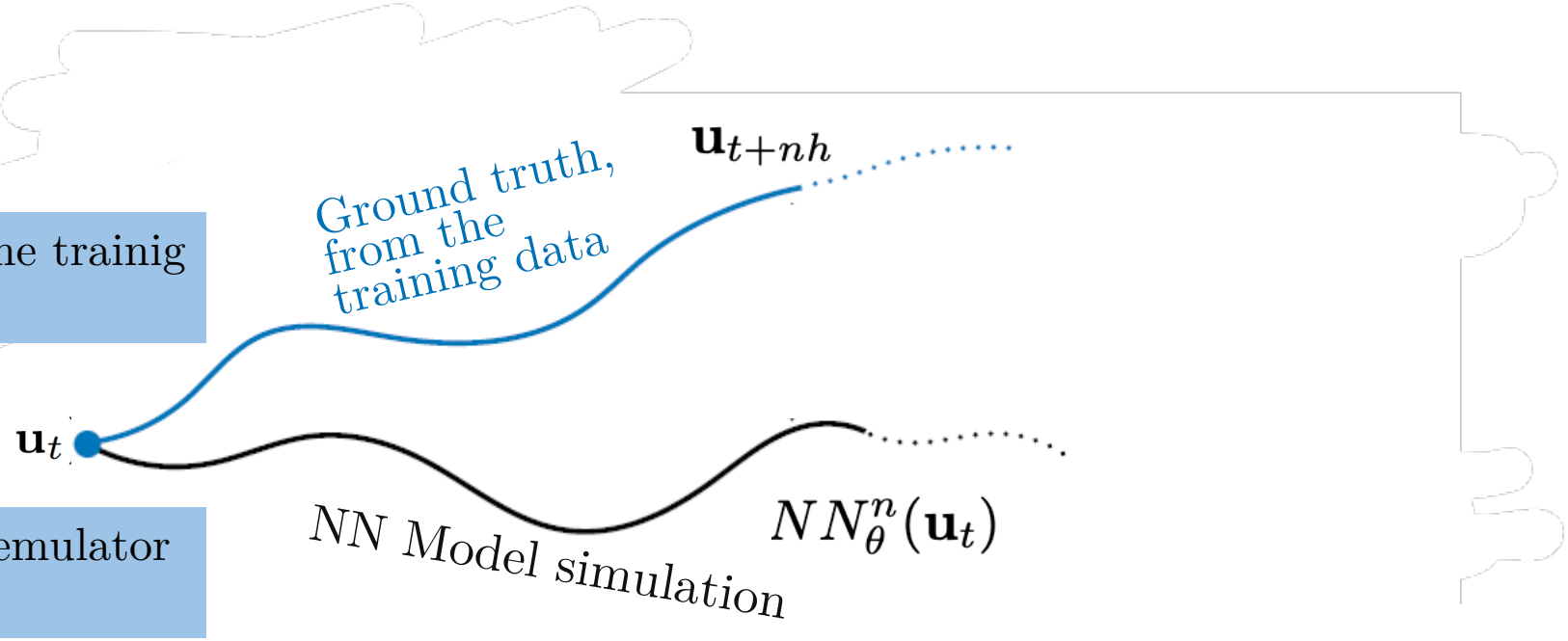


Emulators of numerical models

Training visualization

Initial condition : from the trainig data

Run the autoregressive emulator forward

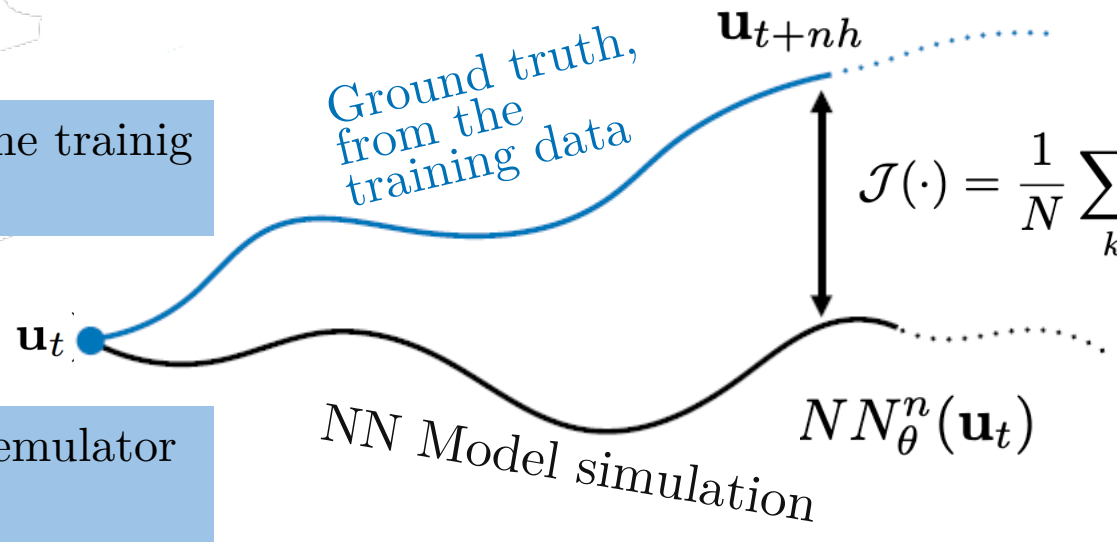


Emulators of numerical models

Training visualization

Initial condition : from the trainig data

Run the autoregressive emulator forward



$$\mathcal{J}(\cdot) = \frac{1}{N}\sum_k \frac{1}{n}\sum_{j=1}^{n} \|NN_\theta^j(\mathbf{u}_{t_k}) - \Psi^j(\mathbf{u}_{t_k})\| + \mathcal{R}(\cdot)$$

Emulators of numerical models

Learning algorithm:

Algorithm 1 Training Autoregressive Emulator from NetCDF Data

Input: NetCDF dataset $\mathcal{D} = \{(\mathbf{s}_t, \mathbf{f}_{t:t+n})\}$, where $\mathbf{s}_t = (\text{state_3d}_t, \text{state_2d}_t)$ is the ocean state and $\mathbf{f}_{t:t+n} = (\tau^x, \tau^y, Q_{\text{net}}, Q_{\text{sol}})_{t:t+n}$ are the atmospheric forcings over the window. Control window size n.
Initialize: Neural Network weights θ
while not converged **do**
    Sample a batch $\mathcal{B} = \{(\mathbf{s}_{t_k}, \mathbf{f}_{t_k:t_k+n}, \mathbf{y}_{t_k+1:t_k+n})\}$ from `train_loader`
    for each sample k in $\mathcal{B}$ **do**
        Set initial state: $\mathbf{z}_0 = \mathbf{s}_{t_k}$
        for $j = 1$ to n **do**
            Retrieve forcing at step $j-1$: $\mathbf{f}_{j-1} = \mathbf{f}_{t_k+j-1}$
            Predict next state: $\mathbf{z}_j = NN_\theta(\mathbf{z}_{j-1}, \mathbf{f}_{j-1})$
            Retrieve ground truth: $\mathbf{y}_j = \mathbf{y}_{t_k+j}$
        end for
    end for
    Compute the trajectory loss $\mathcal{J}(\theta)$:

$$\mathcal{J}(\theta) = \frac{1}{|\mathcal{B}|} \sum_k \frac{1}{n} \sum_{j=1}^{n} \|\mathbf{z}_j - \mathbf{y}_j\|^2 + \mathcal{R}(\theta)$$

    Update θ using an optimizer (e.g., Adam)
end while
Output: Optimized parameters θ

- Data Loading/ Preprocessing/postprocessing
- You should pick and design the neural network model
- Add a regularization if necessary e.g. weight decay
- Implement training/validation logic

Emulators of numerical models

Evaluation procedure:

- Is the emulator fast ? What is the number of SYPD ?
- Is the emulator stable and able to reproduce the long-term variability of the provided data ? ➔ Use the plot functions provided in the github to evaluate if the emulator is able to reproduce correct statistics e.g. climatology, seasonality
- Does the emulator generalize to different forcing conditions ?
- Is the emulator able to generate ensembles (different initial conditions, model configurations, forcings). Evaluate the ensemble spread of the generated ensembles.